

SMS Spam Detection System using NLP and Machine Learning

Project Report

Abstract

The SMS Spam Detection System is a Natural Language Processing (NLP) and Machine Learning based project developed to classify SMS messages as either spam or normal (ham). With the rapid growth of mobile communication, spam messages have become a major issue affecting users worldwide. The primary objective of this project is to automate the detection of spam messages using text classification techniques.

This project applies NLP preprocessing methods such as lowercase conversion and punctuation removal to clean the textual data. The cleaned messages are transformed into numerical vectors using TF-IDF (Term Frequency–Inverse Document Frequency) Vectorization. A Multinomial Naive Bayes classifier is trained on the dataset to predict whether a message is spam or ham.

The model achieved approximately 95% to 98% accuracy, demonstrating effective performance in text classification. This project provides practical understanding of NLP workflow, feature extraction, machine learning model training, and evaluation.

1. Introduction

Natural Language Processing (NLP) is a branch of Artificial Intelligence that enables computers to understand, analyze, and process human language. NLP techniques are widely used in applications such as chatbots, sentiment analysis, spam filtering, recommendation systems, and language translation.

Spam messages are unwanted and harmful messages sent to users for advertisements, fraud, or phishing activities. Detecting spam manually is difficult due to the large number of messages received daily. Therefore, an automated spam detection system is necessary.

The SMS Spam Detection System is designed to classify messages into two categories:

- Spam Message
- Normal (Ham) Message

The system uses Machine Learning algorithms along with NLP preprocessing techniques to analyze and classify messages efficiently.

2. Objective

The main objectives of this project are:

- To develop an NLP-based SMS Spam Detection System.
- To preprocess and clean textual data.
- To convert text into numerical features using TF-IDF Vectorization.
- To train a machine learning model for spam classification.
- To evaluate model performance using accuracy metrics.
- To predict whether a custom message is spam or normal.

3. Problem Statement

Mobile users receive many unwanted spam messages daily. These messages may contain fake advertisements, lottery scams, malicious links, or fraudulent offers. Manual filtering of spam messages is time-consuming and inefficient.

The problem addressed in this project is to automatically identify and classify spam messages using NLP and Machine Learning techniques.

4. Technologies Used

The following technologies and libraries were used in this project:

Technology	Purpose
Python	Programming Language
Pandas	Data Handling and Analysis
NumPy	Numerical Computation
Scikit-learn	Machine Learning Library
NLP	Text Processing
TF-IDF Vectorizer	Feature Extraction
Multinomial Naive Bayes	Classification Algorithm
Jupyter Notebook	Development Environment

5. Dataset Description

The project uses the SMS Spam Collection Dataset containing more than 5,000 SMS messages labeled as spam or ham.

Dataset columns:

Column	Description
label	Indicates spam or ham
message	SMS text message

Example:

label	message
ham	How are you today?
spam	Win cash prize now!

The dataset was loaded using Pandas library.

```
import pandas as pd
```

```
df = pd.read_csv("spam.csv", encoding='latin-1')
```

6. Methodology

The project follows the complete NLP workflow consisting of multiple stages.

6.1 Data Collection

The SMS Spam Collection Dataset was downloaded and stored as a CSV file. The dataset contains labeled SMS messages.

6.2 Data Preprocessing

Text preprocessing is an important step in NLP projects. Raw text data often contains punctuation, uppercase letters, and unwanted symbols.

The following preprocessing steps were applied:

a) Lowercase Conversion

All text messages were converted into lowercase format.

Example:

Before:

WIN MONEY NOW

After:

win money now

Code:

```
df['message'] = df['message'].str.lower()
```

b) Punctuation Removal

Special characters and punctuation marks were removed.

Example:

Before:

Congratulations!!!

After:

congratulations

Code:

```
import string
```

```
df['message'] = df['message'].str.replace(
    f'[{string.punctuation}]',
    "",
    regex=True
)
```

7. Feature Extraction using TF-IDF

Machine Learning models cannot directly understand text data. Therefore, text must be converted into numerical format.

TF-IDF (Term Frequency–Inverse Document Frequency) Vectorization was used for feature extraction.

TF-IDF calculates the importance of words in a document based on frequency.

Formula:

TF-IDF = Term Frequency $\times$ Inverse Document Frequency

Code:

```
from sklearn.feature_extraction.text import TfidfVectorizer
```

```
vectorizer = TfidfVectorizer()
```

```
X = vectorizer.fit_transform(df['message'])
y = df['label']
```

The transformed data becomes a matrix of numerical vectors that can be used for machine learning.

8. Data Splitting

The dataset was divided into training and testing datasets.

- Training Data: 80%
- Testing Data: 20%

Code:

```
from sklearn.model_selection import train_test_split
```

```
X_train, X_test, y_train, y_test = train_test_split(
    X,
    y,
    test_size=0.2,
    random_state=42
)
```

The training dataset is used to train the model, while the testing dataset is used to evaluate performance.

9. Machine Learning Model

The project uses the Multinomial Naive Bayes algorithm.

Why Naive Bayes?

- Fast and efficient
- Works well for text classification
- Suitable for NLP applications
- Performs effectively with TF-IDF vectors

Code:

```
from sklearn.naive_bayes import MultinomialNB
```

```
model = MultinomialNB()
```

```
model.fit(X_train, y_train)
```

The model learns patterns from spam and ham messages during training.

10. Prediction System

The trained model predicts whether a custom message is spam or normal.

Code:

```
msg = ["Hey, are you coming to college today?"]
```

```
msg_vector = tfidf.transform(msg)
```

```
prediction = model.predict(msg_vector)
```

```
print(prediction)
```

Example Output:

Prediction: [0]

Message is Normal

11. Model Evaluation

The model performance was evaluated using Accuracy Score.

Code:

```
from sklearn.metrics import accuracy_score
```

```
prediction = model.predict(X_test)
```

```
accuracy = accuracy_score(y_test, prediction)
```

```
print(accuracy)
```

The model achieved approximately:

95% – 98% Accuracy

This demonstrates that the system effectively classifies spam messages.

12. Advantages of the System

- Automatically detects spam messages
 - Reduces manual filtering effort
 - Fast and efficient prediction
 - High classification accuracy
 - Beginner-friendly NLP implementation
-

13. Limitations

- Model accuracy depends on dataset quality
 - Cannot fully understand message context
 - May misclassify some complex messages
 - Works mainly for English text data
-

14. Future Enhancements

The project can be improved further by:

- Creating a web application using Streamlit or Flask
 - Using Deep Learning models such as LSTM
 - Adding real-time SMS prediction
 - Implementing word cloud visualization
 - Using larger datasets for improved performance
-

15. Learning Outcomes

Through this project, the following concepts were learned:

- Natural Language Processing basics
 - Text preprocessing techniques
 - Feature extraction using TF-IDF
 - Machine Learning model training
 - Spam message classification
 - Model evaluation techniques
 - GitHub project documentation
-

16. Conclusion

The SMS Spam Detection System successfully demonstrates the use of NLP and Machine Learning techniques for text classification. The project preprocesses SMS messages, converts them into numerical vectors using TF-IDF, and classifies them using the Multinomial Naive Bayes algorithm.

The achieved accuracy of approximately 95% to 98% indicates strong model performance. This project provides practical understanding of NLP workflow and serves as a useful beginner-level Machine Learning and NLP application.

References

1. Python Documentation
2. Scikit-learn Documentation
3. Pandas Documentation
4. SMS Spam Collection Dataset
5. Natural Language Processing Concepts